

**CONCEITOS-
-CHAVE**

indicador539

Meta/Questão/Métrica
(Goal/Question/Metric -
GQM).....541

medida538

medição539

métricas

atributos das ...542

código-fonte ...559

modelo

de requisitos....543

orientado a classe .551

projeto da
arquitetura....547projeto da interface
de usuário557projeto orientado
a objeto549projeto para
WebApp557

Um elemento-chave de qualquer processo de engenharia é a medição. Você pode usar medidas para melhorar o entendimento dos atributos dos modelos criados e para avaliar a qualidade dos produtos ou sistemas construídos. Mas diferentemente de outras disciplinas de engenharia, a engenharia de software não é fundamentada nas leis quantitativas da física. Medidas diretas, como tensão (voltagem), massa, velocidade ou temperatura, são incomuns no mundo do software. Devido às medidas e métricas de software serem muitas vezes indiretas, elas estão abertas ao debate. Fenton [Fen91] trata desse assunto quando afirma:

Medição é o processo pelo qual números ou símbolos são anexados aos atributos de entidades no mundo real para defini-los de acordo com regras claramente estabelecidas... Nas ciências físicas, medicina, economia e mais recentemente nas ciências sociais, podemos medir os atributos antes considerados incomensuráveis... É claro que essas medidas não são tão refinadas como muitas feitas nas ciências físicas, mas existem [e são tomadas decisões importantes baseadas nelas]. Percebemos que a obrigação de tentar “medir o incomensurável” para melhorar nossa compreensão de entidades particulares é tão poderosa na engenharia de software quanto em qualquer outra disciplina.

Mas alguns membros da comunidade de software continuam a argumentar que o software é “incomensurável” ou que tentativas de medições deverão ser adiadas até entendermos melhor o software e os atributos que deverão ser usados para descrevê-lo. Isso é um erro.

PANORAMA

O que é? Por sua natureza, a engenharia é uma disciplina quantitativa. A métrica de produto ajuda os engenheiros de software a visualizar o projeto e a construção do software, focalizando atributos específicos e mensuráveis dos artefatos da engenharia de software.

Quem realiza? Os engenheiros de software usam métricas de produto para ajudá-los a criar software de mais alta qualidade.

Por que é importante? Haverá sempre um elemento qualitativo na criação de software. O problema é que a avaliação qualitativa pode não ser suficiente. É preciso critérios objetivos para ajudar a direcionar o projeto de dados, arquitetura, interfaces e componentes. Ao testar, necessitamos de orientação quantitativa que nos auxiliará na seleção de casos de teste e seus objetivos. A métrica de produto proporciona uma base por meio da qual a análise, projeto, codificação e teste podem ser conduzidos mais objetivamente e avaliados de maneira mais quantitativa.

Quais são as etapas envolvidas? O primeiro passo no processo de medição é derivar as

medições de software e as métricas apropriadas para a representação do software. Em seguida, coletam-se os dados necessários para derivar as métricas formuladas. Uma vez computadas, as métricas apropriadas são analisadas com base em diretrizes preestabelecidas e dados do passado. Os resultados das análises são interpretados para obter informações sobre a qualidade do software e os dados da interpretação levam à modificação dos requisitos e modelos de projeto, código-fonte ou casos de teste. Em algumas instâncias, pode também levar à modificação do próprio processo de software.

Qual é o artefato? As métricas de produto computadas por meio de dados coletados dos requisitos e modelos de projeto, código-fonte e casos de teste.

Como garantir que o trabalho foi realizado corretamente? Devemos estabelecer os objetivos da medição antes de iniciarmos a coleta de dados, definindo cada métrica de produto de maneira não ambígua. Defina apenas algumas métricas e então as use para obter informações sobre a qualidade de um artefato de software.

teste	560
ponto de função (FP)	543
princípios de medição	540

Embora as métricas de produto para programas de computadores sejam imperfeitas, podem proporcionar uma maneira sistemática de avaliar a qualidade com base em um conjunto de regras claramente definidas. Elas também proporcionam uma visão objetiva, que “vai direto ao ponto” e não “após o fato”. Isso permite descobrir e corrigir problemas potenciais antes que se tornem defeitos catastróficos.

Neste capítulo, apresentam-se medidas que podem ser usadas para avaliar a qualidade do produto enquanto está sendo projetado. Essas medidas de atributos internos do produto fornecem uma indicação em tempo real da eficácia dos modelos de requisitos, projeto e código; a eficácia dos casos de teste; e a qualidade geral do software que será criado.

23.1 ESTRUTURA PARA MÉTRICAS DE PRODUTO

“Uma ciência é tão desenvolvida quanto suas ferramentas de medição.”

Louis Pasteur

Conforme mencionado na introdução, a medição atribui números ou símbolos a atributos de entidades no mundo real. Para tanto, é necessário um modelo de medição abrangendo um conjunto consistente de regras. Embora a teoria da medição (por exemplo, [Kyb84]) e sua aplicação a programas de computadores (por exemplo, [Zus97]) sejam tópicos que estão além do escopo deste livro, vale estabelecer uma estrutura fundamental e um conjunto de princípios básicos que orientem a definição de métricas de produto para software.

23.1.1 Medidas, métricas e indicadores

? Qual a diferença entre uma medida e uma métrica?

Embora os termos *medida*, *medição* e *métricas* sejam usados com frequência de forma intercambiável, é importante notar as diferenças sutis entre eles. Como *medida* pode ser usada como substantivo ou como verbo, as definições do termo podem se tornar confusas. Sob o contexto de engenharia de software, *medida* proporciona uma indicação quantitativa da extensão, quantidade, capacidade ou tamanho de algum atributo de um produto ou processo. *Medição* é o ato de determinar uma medida. O *IEEE Standard Glossary of Software Engineering Terminology* [IEE93b] define *métrica* como “uma medida quantitativa do grau com o qual um sistema, componente ou processo possui determinado atributo.”

Quando é coletado um único ponto de dado (por exemplo, o número de erros descobertos em um componente de software), foi estabelecida uma medida. A medição ocorre como resultado da coleção de um ou mais pontos de dados (por exemplo, um conjunto de revisões de componente e testes de unidade são investigados para coletar medidas do número de erros para cada um). Uma métrica de software relaciona as medidas individuais de alguma maneira (por exemplo, o número médio de erros encontrados por revisão ou o número médio de erros encontrados por teste de unidade).

Um engenheiro de software coleta medidas e desenvolve métricas para obter indicadores. Um *indicador* é uma métrica ou combinação de métricas que proporcionam informações sobre o processo de software, em um projeto de software ou no próprio produto. Um indicador proporciona informações que permitem ao gerente de projeto ou aos engenheiros de software ajustar o processo, o projeto ou o produto para incluir melhorias.

23.1.2 O desafio das métricas de produto

Durante as últimas quatro décadas, muitos pesquisadores tentaram desenvolver uma única métrica que forneça uma medida fácil de entender a complexidade do software. Fenton [Fen94] caracteriza essa pesquisa como uma busca do “impossível santo graal”. Embora tenham sido propostas dezenas de medidas de complexidade [Zus90], cada uma delas tem uma visão diferente do que é a complexidade e quais os atributos de um sistema que levam à complexidade. Por analogia, considere uma métrica para avaliar um carro de luxo. Alguns observadores podem enfatizar o projeto da carroceria; outros considerar as características mecânicas; outros ainda podem considerar o custo ou desempenho ou o uso de combustíveis alternativos ou a facilidade

PONTO-CHAVE

Um indicador é uma métrica ou métricas que proporcionam visão do processo, produto ou do projeto.

“Assim como a medição da temperatura começou com um dedo indicador... E chegou até as escalas, instrumentos e técnicas sofisticadas, a medição de software também está evoluindo.”

Shari Pfleeger

de reciclagem quando o carro já estiver inutilizável. Como cada uma dessas características pode ser estranha em relação às outras, é difícil derivar um valor único para “atraente”. O mesmo problema ocorre com o software.

No entanto, há uma necessidade de medir e controlar a complexidade do software. E, se é difícil obter um valor único dessa métrica de qualidade, deverá ser possível desenvolver medidas de diferentes atributos internos de programa (por exemplo, modularidade efetiva, independência funcional e outros discutidos no Capítulo 8). Essas medidas e métricas derivadas dos atributos podem ser usadas como indicadores independentemente da qualidade dos modelos de requisitos e projeto. Mas aqui também temos problemas. Fenton [Fen94] observa isso quando diz: “O perigo de tentar encontrar medidas que caracterizem tantos atributos diferentes é que inevitavelmente as medidas têm de satisfazer interesses em conflito. Isso é contrário à teoria representacional da medição”. Embora a afirmação de Fenton esteja correta, muitos argumentam que a medição de produto executada durante os primeiros estágios do processo de software fornece aos engenheiros um mecanismo consistente e objetivo para avaliar a qualidade.

É justo questionar, no entanto, quão válidas são as métricas de produto. Quanto as métricas de produto estão alinhadas à confiabilidade e qualidade de longo prazo para um sistema de computador? Fenton [Fen91] trata a questão da seguinte maneira:

Apesar das conexões intuitivas entre a estrutura interna dos artefatos [métricas de produto] e seus atributos externos de produto e processo, tem havido na realidade poucas tentativas científicas de estabelecer relações específicas. Há várias razões para isso; a mais citada é a impraticabilidade de executar experimentos relevantes.

Cada um dos “desafios” mencionados aqui é motivo para cuidado, mas não há por que dispensar as métricas de produto.¹ A medição é essencial se o que se deseja é qualidade.

23.1.3 Princípios de medição

Antes de introduzirmos uma série de métricas de produto que (1) ajuda na avaliação dos modelos de análise e projeto, (2) proporciona uma indicação da complexidade dos projetos procedimentais e código-fonte e (3) facilita o projeto de testes mais eficazes, é importante entendermos os princípios básicos de medições. Roche [Roc94] sugere um processo de medição que pode ser caracterizado por cinco atividades:

- *Formulação.* A criação de medidas e métricas de software apropriadas para a representação do software considerado.
- *Coleção.* O mecanismo usado para acumular dados necessários para criar as métricas formuladas.
- *Análise.* A computação das métricas e a aplicação de ferramentas matemáticas.
- *Interpretação.* A avaliação de métricas que resultam em informações sobre a qualidade da representação.
- *Feedback.* Recomendações derivadas da interpretação de métricas de produto transmitidas para a equipe de software.

As métricas de software serão úteis somente se forem efetivamente caracterizadas e validadas de forma que demonstrem valer a pena. Os princípios a seguir [Let03b] são representativos de muitos que podem ser propostos para caracterização métrica e validação:

- *Uma métrica deve ter as propriedades matemáticas desejadas.* O valor da métrica deverá estar em um intervalo significativo (por exemplo, 0 a 1, em que 0 significa ausência, 1 indica

WebRef

Um grande volume de informações sobre métricas de produto foi compilado por Horst Zuse em irb.cs.tu-berlin.de/~zuse/.

? Quais os passos para um processo eficaz de medição?

¹ Embora seja comum na literatura a crítica a métricas específicas, muitas críticas focalizam aspectos secretos e perdem o objetivo primário das métricas no mundo real: ajudar o engenheiro de software a estabelecer uma maneira sistemática e objetiva de visualizar seu trabalho e melhorar a qualidade do produto como resultado.



Na realidade, muitas métricas de produto em uso atualmente não se enquadram nesses princípios tão bem quanto deveriam. Mas isso não significa que elas não têm valor — apenas seja cuidadoso ao usá-las, entendendo que se destinam a proporcionar informações, não uma rígida verificação científica.

o valor máximo e 0,5 representa o “ponto médio”). Além disso, uma métrica que deve estar em uma escala racional não deve ser composta de componentes medidos apenas em escala ordinal.

- Quando uma métrica representa uma característica do software que aumenta na ocorrência de peculiaridades positivas ou diminui na ocorrência de peculiaridades indesejadas, o valor da métrica deverá aumentar ou diminuir da mesma maneira.
- Cada métrica deverá ser validada empiricamente em uma ampla variedade de contextos antes de ser publicada ou usada para tomada de decisões. Uma métrica deverá medir o fator de interesse, independentemente de outros fatores. Ela deve ser “ampliada” para sistemas grandes e funcionar em uma variedade de linguagens de programação e domínios de sistemas.

Embora a formulação, caracterização e validação sejam de extrema importância, a coleta e a análise são as atividades que regem o processo de medição. Roche [Roc94] sugere os seguintes princípios para essas atividades: (1) sempre que possível, a coleta e análise de dados deverá ser automática; (2) deverão ser aplicadas técnicas estatísticas válidas para estabelecer relações entre atributos internos de produto e características externas de qualidade (por exemplo, se o nível de complexidade da arquitetura está correlacionado ao número de defeitos relatados em uso); e (3) diretrizes e recomendações interpretativas deverão ser estabelecidas para cada métrica.

23.1.4 Medição de software orientada a objetivo

O paradigma *Meta/Questão/Métrica* (*Goal/Question/Metric* – GQM) foi desenvolvido por Basili e Weiss [Bas84] como uma técnica para identificar métricas significativas para qualquer parte do processo de software. O GQM enfatiza a necessidade de (1) estabelecer um *objetivo* de medição explícita que é específico para a atividade do processo ou característica de produto que deve ser avaliada, (2) definir um conjunto de *questões* que devem ser respondidas para atingir o objetivo e (3) identificar *métricas* bem formuladas que ajudam a responder a essas questões.

Pode ser usado um *modelo de definição de objetivo* (*goal definition template*) [Bas94] para definir o objetivo de cada medição. O modelo toma a seguinte forma:

Analisar {o nome da atividade ou atributo a ser medido} **com a finalidade de** {o objetivo geral da análise²} **com relação a** {o aspecto da atividade ou atributo considerado} **do ponto de vista de** {a pessoa que tem o interesse na medição} **no contexto de** {o ambiente no qual a medição ocorre}.

Como exemplo, considere um modelo de definição de objetivo para o *CasaSegura*:

Analisar a arquitetura de software do *Casa Segura* **com a finalidade de** avaliar componentes da arquitetura **com relação à** habilidade em tornar o *CasaSegura* mais ampliável **do ponto de vista dos** engenheiros de software que estão executando o trabalho **no contexto de** aperfeiçoamento de produto durante os próximos três anos.

Com o objetivo de medição explicitamente definido, desenvolve-se uma série de questões. As respostas a essas questões ajudam a equipe de software (ou outros interessados) a determinar se o objetivo de medição foi atingido. Entre as questões que podem ser formuladas estão:

- Q₁: Os componentes de arquitetura são caracterizados para distinguir função de dados relacionados?
- Q₂: A complexidade de cada componente está dentro dos limites que facilitarão a modificação e extensão?

WebRef

Uma discussão útil sobre GQM pode ser encontrada em www.thedacs.com/GoldPractices/practices/gqma.html.

² van Solingen e Berghout [Sol99] sugerem que o objetivo é quase sempre “entender, controlar ou improvisar” a atividade de processo ou atributo de produto.

Cada uma dessas questões deverá ser respondida quantitativamente, usando uma ou mais medidas e métricas. Por exemplo, uma métrica que proporciona uma indicação da coesão (Capítulo 8) de um componente de arquitetura pode ser útil na resposta de Q_1 . Métricas discutidas mais adiante neste capítulo podem proporcionar uma visão para Q_2 . Em todos os casos, as métricas escolhidas (ou derivadas) deverão estar de acordo com os princípios de medição discutidos na Seção 23.1.3 e os atributos de medição discutidos na Seção 23.1.5.

23.1.5 Atributos de métricas eficazes de software

Centenas de métricas já foram propostas para programas de computadores, mas nem todas proporcionam suporte prático ao engenheiro de software. Algumas demandam medições muito complexas, outras são tão esotéricas que poucos profissionais do mundo real têm qualquer esperança de entendê-las, e outras ainda violam as noções intuitivas básicas do que é realmente um software de alta qualidade.

Ejiogu [Eji91] define um conjunto de atributos que deverão ser abrangidos por métricas, de software efetivas. A métrica derivada e as medições que levam até ela deverão ser:

 **Como devemos avaliar a qualidade de uma métrica de software proposta?**

- *Simples e computáveis.* Deverá ser relativamente fácil aprender a derivar a métrica, e sua computação não deve demandar esforço ou tempo fora do normal.
- *Empiricamente e intuitivamente persuasiva.* A métrica deverá satisfazer as ideias do engenheiro sobre o atributo do produto considerado (por exemplo, uma métrica que mede coesão de módulo deverá crescer em valor na medida em que aumenta o nível de coesão).
- *Consistente e objetiva.* A métrica deverá sempre produzir resultados que não sejam ambíguos. Um terceiro independente deverá ser capaz de derivar o mesmo valor da métrica usando as mesmas informações sobre o software.
- *Consistente no seu uso das unidades e dimensões.* A computação matemática da métrica deverá usar medidas que não resultem em combinações bizarras de unidades. Por exemplo, multiplicar número de pessoas nas equipes de projeto pelas variáveis da linguagem de programação no programa resulta em uma mistura duvidosa de unidades que não é claramente convincente.
- *Independente da linguagem de programação.* As métricas deverão ser baseadas no modelo de requisitos, modelo de projeto ou na própria estrutura do programa. Elas não deverão ser dependentes dos caprichos da sintaxe ou semânticas das linguagens de programação.
- *Um mecanismo efetivo para feedback de alta qualidade.* A métrica deverá fornecer informações que podem levar a um produto final de melhor qualidade.



A experiência indica que a métrica de um produto será usada somente se ela for clara e fácil de computar. Se forem necessárias dezenas de “contagens” e computações complexas, é pouco provável que seja amplamente adotada.

Embora muitas métricas de software satisfaçam a esses atributos, algumas usadas comumente podem não satisfazer a um ou outro desses atributos. Um exemplo é o ponto de função (*function point* – FP) (discutida na Seção 23.2.1) – uma medida da “funcionalidade” fornecida pelo software. Alguém pode argumentar³ que o atributo consistente e objetivo falha porque um terceiro, independente, pode não ser capaz de derivar o mesmo valor ponto de função que um colega usando as mesmas informações sobre o software. Deveremos então rejeitar a medida FP? A resposta é “claro que não!” A métrica FP proporciona informações úteis e valor distinto, mesmo que não satisfaça a um atributo perfeitamente.

3 Pode-se apresentar um contra-argumento igualmente forte. Essa é a natureza da métrica de software.

CASA SEGURA



Debatendo métricas de produto

Cena: Sala do Vinod.

Atores: Vinod, Jamie e Ed — membros da equipe de engenharia de software do CasaSegura dando segmento ao trabalho de projeto em nível de componente e projeto de casos de teste.

Conversa:

Vinod: Doug [Doug Miller, gerente de engenharia de software] me disse que todos nós devemos usar métricas de produto, mas ele foi um pouco vago. Ele falou também que não vai “forçar a barra”... Usar ou não, depende de nós.

Jamie: Ótimo, porque não tenho como arranjar tempo para começar a medir coisas. Estamos lutando para manter o cronograma.

Ed: Concordo com o Jamie. Somos contra, aqui... Não temos tempo.

Vinod: Sim, eu sei, mas provavelmente deve haver algum mérito em usar tais métricas.

Jamie: Não estou discutindo isso, Vinod, é a questão do tempo... E nenhum de nós tem qualquer disponibilidade.

Vinod: Mas se essas medições ajudarem a poupar o seu tempo?

Ed: Errado, demandam horas livres e como disse o Jamie...

Vinod: Não, espere... E se isso nos ajudar a poupar tempo?

Jamie: Como?

Vinod: Retrabalho... É isso. Se uma medição que fizermos nos ajudar a evitar um problema maior ou mesmo moderado, e se isso impedir o retrabalho em uma parte do sistema, nos poupará tempo. Não?

Ed: É possível, suponho, mas você pode garantir que alguma métrica de produto nos ajudará a encontrar um problema?

Vinod: Você pode garantir que não?

Jamie: Então o que você propõe?

Vinod: Acho que poderíamos selecionar algumas métricas de projeto, provavelmente orientadas a classe, e usá-las como parte de nosso processo de revisão para qualquer componente que desenvolvermos.

Ed: Não estou familiarizado com métricas orientadas a classe.

Vinod: Vou verificar tudo isso e fazer uma recomendação... Ok com vocês pessoal?

[Ed e Jamie balançam a cabeça sem muito entusiasmo.]

23.2 MÉTRICAS PARA O MODELO DE REQUISITOS

O trabalho técnico na engenharia de software começa com a criação do modelo de requisitos. É nesse estágio que os requisitos são formulados e é estabelecida uma base para o projeto. Portanto, métricas de produto que proporcionem informações sobre a qualidade do modelo de análise são desejáveis.

Embora relativamente poucas métricas de análise e especificação tenham aparecido na literatura, é possível adaptar as usadas frequentemente para estimativa de projeto e aplicá-las nesse contexto. Essas métricas examinam o modelo de requisitos com a intenção de prever o “tamanho” do sistema resultante. O tamanho é às vezes (mas nem sempre) um indicador da complexidade do projeto e quase sempre é um indicador do trabalho cada vez maior de codificação, integração e teste.

23.2.1 Métricas baseadas em função

A *métrica ponto de função* pode ser usada efetivamente como um meio para medir a funcionalidade fornecida por um sistema.⁴ Por meio de dados históricos, a métrica FP pode ser empregada para (1) estimar o custo ou trabalho necessário para projetar, codificar e testar o software; (2) prever o número de erros que serão encontrados durante o teste; e (3) prever o número de componentes e/ou o número de linhas projetadas de código-fonte no sistema implementado.

Pontos de função são derivados por meio de uma relação empírica baseada em medidas calculáveis (diretas) do domínio de informações do software e avaliações qualitativas da complexidade do software. Valores do domínio de informações são definidos da seguinte maneira:⁵

WebRef

Muitas informações úteis sobre ponto de função podem ser obtidas em www.ifpug.org e www.functionpoints.com.

⁴ Centenas de livros, dissertações e artigos já foram escritos sobre métricas FP. Uma boa bibliografia pode ser encontrada em [IFP05].

⁵ Atualmente, a definição dos valores do domínio de informações e a maneira pela qual elas são contadas é algo um pouco mais complexo. O leitor interessado deve consultar [IFP01] para mais detalhes.

Número de entradas externas (*number of external inputs – EEs*). Cada *entrada externa* é originada de um usuário ou transmitida de outra aplicação e fornece dados distintos orientados a aplicação ou informações de controle. Entradas são muitas vezes usadas para atualizar *arquivos lógicos internos* (*internal logical files – ILFs*). As entradas devem ser diferenciadas das consultas, que são contadas separadamente.

Número de saídas externas (*number of external outputs – EOs*). Cada *saída externa* é formada por dados derivados da aplicação e fornece informações para o usuário. Nesse contexto, as saídas externas se referem a relatórios, telas, mensagens de erro etc. Itens individuais de dados em um relatório não são contados separadamente.

Número de consultas externas (*number of external inquiries – EQs*). Uma *consulta externa* é definida como uma entrada on-line que resulta na geração de alguma resposta imediata do software na forma de uma saída on-line (muitas vezes obtida de um ILF).

Número de arquivos lógicos internos (*number of internal logical files – ILFs*). Cada *arquivo lógico interno* é um agrupamento lógico de dados que reside dentro das fronteiras do aplicativo e é mantido através de entradas externas.

Número de arquivos de interface externos (*number of external interface files – EIFs*). Cada *arquivo de interface externo* é um agrupamento lógico de dados que reside fora da aplicação, mas fornece informações que podem ser usadas pela aplicação.

Uma vez coletados esses dados, é completada a tabela da Figura 23.1 e associado um valor de complexidade com cada contagem. Organizações que usam métodos ponto de função desenvolvem critérios para determinar se determinada entrada é simples, média ou complexa. No entanto, a determinação da complexidade é de certo modo subjetivo.

Para calcular pontos de função (FP), usa-se a seguinte relação:

$$FP = \text{contagem total} \times [0,65 + 0,01 \times \sum (F_i)] \quad (23.1)$$

em que a contagem total é a soma de todas as entradas FP obtidas da Figura 23.1.

Os F_i ($i = 1$ a 14) são *fatores de ajuste de valor* (*value adjustment factors – VAF*) baseados em respostas às questões a seguir [Lon02]:

1. O sistema requer salvamento (*backup*) e recuperação confiável (*recovery*)?
2. São necessárias comunicações de dados especializadas para transferir informações para a aplicação ou da aplicação?
3. Há funções de processamento distribuído?

PONTO-CHAVE

Os fatores de ajuste de valor são usados para fornecer uma indicação da complexidade do problema.

FIGURA 23.1

Computando pontos de função

Valor do Domínio de Informação	Contagem		Fator de peso			
			Simple	Médio	Complexo	
Entradas externas (EIs)		×	3	4	6	=
Saídas Externas (EOs)		×	4	5	7	=
Consultas Externas (EQs)		×	3	4	6	=
Arquivos Lógicos Internos (ILFs)		×	7	10	15	=
Arquivos de Interface Externos (EIFs)		×	5	7	10	=
Contagem total						→

4. O desempenho é crítico?
5. O sistema rodará em um ambiente operacional existente e intensamente utilizado?
6. O sistema requer entrada de dados on-line?
7. A entrada on-line de dados requer que a transação de entrada seja composta em múltiplas telas ou operações?
8. Os ILFs são atualizados on-line?
9. As entradas, saídas, arquivos ou consultas são complexas?
10. O processamento interno é complexo?
11. O código é projetado para ser reutilizável?
12. A conversão e instalação estão incluídas no projeto?
13. O sistema é projetado para múltiplas instalações em diferentes organizações?
14. A aplicação é projetada para facilitar a troca e o uso pelo usuário?

WebRef

Há uma calculadora on-line para FP em irb.cs.unimagdeburg.de/sw-eng/us/java/tp/.

Cada uma dessas perguntas é respondida por meio de uma escala que varia de 0 (não importante ou não aplicável) a 5 (absolutamente essencial). Os valores das constantes na Equação (23.1) e os fatores de peso aplicados aos valores do domínio de informações são determinados empiricamente.

Para ilustrarmos o uso da métrica FP nesse contexto, consideramos a representação de um modelo simples de análise, apresentado na Figura 23.2. Na figura, está representado um diagrama de fluxo de dados (Capítulo 7) para uma função do software *CasaSegura*.

A função gerencia a interação do usuário, aceitando uma senha para ativar ou desativar o sistema e possibilita consultas sobre o estado das zonas de segurança e vários sensores de segurança. A função mostra uma série de mensagens imediatas e envia sinais de controle apropriados para os vários componentes do sistema de segurança.

O diagrama de fluxo de dados é avaliado para determinar um conjunto-chave de medidas de domínio de informação necessárias para a computação da métrica ponto de função. Três entradas externas — **senha**, **botão de emergência** e **ativar/desativar** — são exibidas na figura com duas consultas externas — **consulta de zona** e **consulta de sensor**. É mostrado um ILF (**arquivo de configuração de sistema**). São apresentadas também duas saídas externas (**mensagens** e **estado do sensor**) e quatro EIFs (**sensor de teste**, **configuração de zona**, **ativar/desativar** e **alerta de alarme**). Esses dados, com sua complexidade apropriada, estão na Figura 23.3.

O total da contagem mostrado na Figura 23.3 deve ser ajustado usando a Equação (23.1). Para as finalidades desse exemplo, supomos que $\sum (F_i)$ é 46 (um produto moderadamente complexo). Portanto,

$$FP = 50 \times [0,65 \times (0,01 \times 46)] = 56$$

FIGURA 23.2

Um modelo de fluxo de dados para o software *CasaSegura*

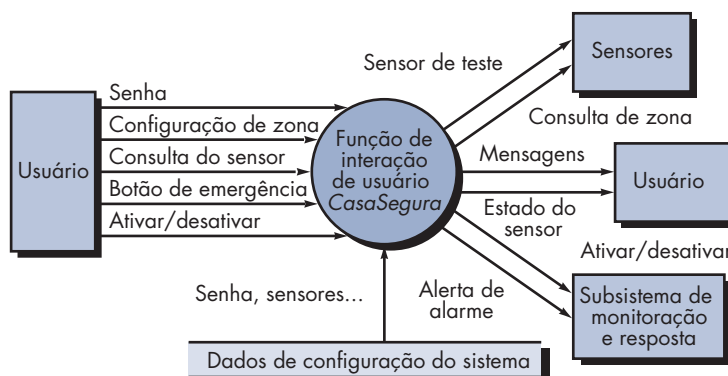


FIGURA 23.3

Computando pontos de função

Valor do Domínio de Informação	Contagem		Fator de peso			
			Simples	Médio	Complexo	
Entradas externas (EIs)	3	×	3	4	6	= 9
Saídas Externas (EOs)	2	×	4	5	7	= 8
Consultas Externas (EQs)	2	×	3	4	6	= 6
Arquivos Lógicos Internos (ILFs)	1	×	7	10	15	= 7
Arquivos de Interface Externos (EIFs)	4	×	5	7	10	= 20
Contagem total						50

“Em vez de ficar apenas meditando sobre “para que serve a nova métrica”... Deveríamos fazer também a pergunta mais básica, “O que vamos fazer com as métricas?”

Michael Mah e Larry Putnam

Com base no valor FP projetado, derivado do modelo de requisitos, a equipe de projeto pode estimar o tamanho total implementado da função de interação de usuário do *CasaSegura*. Vamos assumir que dados já conhecidos indicam que um FP se traduz em 60 linhas de código (será usada uma linguagem orientada a objeto) e que 12 FPs são produzidos por cada pessoa-mês de trabalho. Esses dados históricos fornecem ao gerente do projeto informações importantes de planejamento baseadas no modelo de requisitos e não nas estimativas preliminares. Suponha ainda que projetos anteriores tenham apresentado uma média de três erros por ponto de função durante as revisões de requisitos e projeto e quatro erros por ponto de função durante o teste de unidade e integração. Esses dados podem finalmente ajudá-lo a avaliar a totalidade das suas atividades de revisão e teste.

Uemura e seus colegas [Uem99] sugerem que pontos de função podem também ser computados por meio dos diagramas de classe UML e sequência. Se você tiver mais interesse, veja os detalhes em [Uem99].

23.2.2 Métricas para qualidade de especificação

Davis e seus colegas [Dav93] propõem uma lista de características que podem ser usadas para avaliar a qualidade do modelo de requisitos e as especificações de requisitos correspondentes: *peculiaridade* (ausência de ambiguidade), *totalidade*, *exatidão*, *entendimento*, *repetibilidade*, *consistência interna e externa*, *disponibilidade*, *brevidade*, *rastreabilidade*, *modificação*, *precisão* e *reutilização*. Além disso, os autores observam que há especificações de alta qualidade armazenadas eletronicamente; executáveis ou pelo menos interpretáveis; comentadas de acordo com sua importância relativa; e estáveis versões atualizadas, organizadas referências cruzadas e especificadas com o nível correto de detalhe.

Embora muitas dessas características pareçam ser qualitativas em sua natureza, Davis et al. [Dav93] sugerem que cada uma delas pode ser representada usando uma ou mais métricas. Por exemplo, assumimos que há nr requisitos em uma especificação, tal que

$$n_r = n_f + n_{nf}$$

em que n_f é o número de requisitos funcionais e n_{nf} é o número de requisitos não funcionais (por exemplo, desempenho).

Para determinarem a *peculiaridade* (ausência de ambiguidade) dos requisitos, Davis et al. sugerem uma métrica baseada na consistência da interpretação de cada requisito por parte dos revisores:

$$Q_1 = \frac{n_{ui}}{n_r}$$

PONTO-CHAVE

Medindo-se características da especificação, é possível obter informações quantitativas sobre peculiaridade e totalidade.

em que n_{ui} é o número de requisitos para os quais todos os revisores tiveram interpretações idênticas. Quanto mais próximo o valor Q estiver de 1, mais baixa será a ambiguidade da especificação.

A *totalidade* dos requisitos funcionais pode ser determinada calculando-se a relação

$$Q_2 = \frac{n_u}{n_i \times n_s}$$

em que n_u é o número de requisitos funcionais únicos, n_i é o número de entradas (estímulos) definidas ou implícitas pela especificação e n_s é o número de estados especificados. A relação Q_2 mede a porcentagem de funções necessárias especificadas para um sistema. No entanto, ela não trata de requisitos não funcionais. Para incorporar esses requisitos em uma métrica global para a totalidade, você deve considerar o grau com o qual os requisitos foram validados:

$$Q_3 = \frac{n_c}{n_c + n_{nv}}$$

em que n_c é o número de requisitos validados como corretos e n_{nv} é o número de requisitos que não foram ainda validados.

“Meça aquilo que é mensurável, e o que não for torne mensurável.”

Galileu

23.3 MÉTRICAS PARA O MODELO DE PROJETO

PONTO-CHAVE

As métricas podem proporcionar informações sobre dados estruturais e complexidade do sistema associadas ao projeto da arquitetura.

É inconcebível que o projeto de um novo avião, um novo chip de computador ou um novo edifício comercial seja conduzido sem a definição de medidas, sem determinar métricas para os vários aspectos de qualidade e sem usá-las como indicadores para orientar a maneira pela qual o projeto evoluirá. E, além disso, o projeto de sistemas complexos baseados em software muitas vezes é realizado praticamente sem nenhuma medição. A ironia disso tudo é que as métricas de projeto para software estão disponíveis, mas a grande maioria dos engenheiros de software continua ignorando sua existência.

As métricas de projeto para software de computador, como todas as outras métricas de software, não são perfeitas. Continua o debate sobre sua eficácia e a maneira pela qual deveriam ser aplicadas. Muitos especialistas argumentam que é necessária mais experimentação para que as medições de projeto possam ser usadas. E, além disso, projeto sem medição é uma alternativa inaceitável.

Nas próximas seções, examinaremos algumas das métricas de projeto mais comuns para software de computador. Cada uma delas pode proporcionar uma melhor visualização, e todas podem ajudar o projeto a evoluir para um nível maior de qualidade.

23.3.1 Métricas de projeto da arquitetura

As métricas de projeto da arquitetura focalizam as características da arquitetura do programa (Capítulo 9) com ênfase na estrutura arquitetônica e na eficácia dos módulos ou componentes dentro da arquitetura. Essas métricas são uma “caixa-preta” no sentido de que elas não requerem qualquer conhecimento do funcionamento interno de um determinado componente de software.

Card e Glass [Car90] definem três medidas de complexidade de projeto de software: complexidade estrutural, complexidade de dados e complexidade de sistema.

Para arquiteturas hierárquicas (por exemplo, arquiteturas de chamada e retorno), a *complexidade estrutural* de um módulo i é definida da seguinte maneira:

$$S(i) = f_{out}^{(i)2}$$

em que $f_{out}^{(i)}$ é o fan-out⁶ do módulo i .

6 Fan-out é definido como o número de módulos imediatamente subordinados ao módulo i ; o número de módulos chamados diretamente pelo módulo i .

A *complexidade dos dados* (*data complexity*) proporciona uma indicação da complexidade na interface interna para um módulo i e é definida como

$$D(i) = \frac{v(i)}{f_{\text{out}}(i) + 1}$$

em que $v(i)$ é o número de variáveis de entrada e saída passadas para e do módulo i .

Por fim, a *complexidade do sistema* (*system complexity*) é definida como a soma da complexidade estrutural e de dados, especificada como

$$C(i) = S(i) + D(i)$$

À medida que esses valores de complexidade aumentam, a complexidade global da arquitetura do sistema também aumenta. Isso leva a uma maior probabilidade de que o trabalho de integração e teste também aumente.

Fenton [Fen91] sugere um conjunto de métricas de morfologia simples (isto é, forma) que permite que diferentes arquiteturas de programa sejam comparadas usando uma série de dimensões diretas. Referindo-se à arquitetura de chamada e retorno na Figura 23.4, podem ser definidas as seguintes métricas:

$$\text{Tamanho} = n + a$$

em que n é o número de nós e a é o número de arcos. Para a arquitetura mostrada na Figura 23.4,

$$\text{Tamanho} = 17 + 18 = 35$$

Profundidade = caminho mais longo desde o nó raiz (topo) até o nó da folha. Para a arquitetura mostrada na Figura 23.4, profundidade = 4.

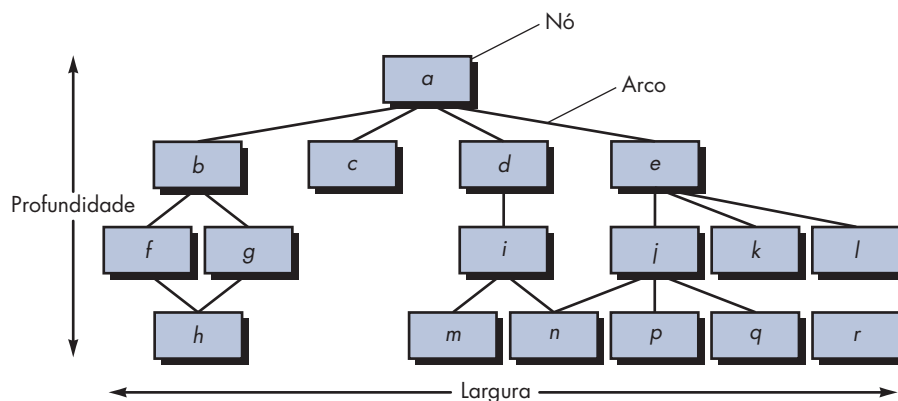
Largura = número máximo de nós em qualquer nível da arquitetura. Para a arquitetura mostrada na Figura 23.4, largura = 6.

A relação arco para nó, $r = a/n$, mede a densidade de conectividade da arquitetura e pode fornecer uma indicação simples do acoplamento da arquitetura. Para a arquitetura mostrada na Figura 23.4, $r = 18 / 17 = 1,06$.

A força aérea americana (U.S. Air Force Systems Command) [USA87] desenvolveu um conjunto de indicadores de qualidade de software baseados nas características de projeto mensuráveis de um programa de computador. Usando conceitos similares àqueles propostos na norma IEEE Std. 982.1-1988 [IEE94], a Força Aérea usa informações obtidas do projeto de dados e de

FIGURA 23.4

Métricas de morfologia



arquitetura para criar um *índice de qualidade de projeto de estrutura* (design structure quality index – DSQI) que varia de 0 a 1. Os seguintes valores devem ser apurados para computar o DSQI [Cha89]:

- S_1 = número total de módulos definidos na arquitetura do programa
- S_2 = número de módulos cuja função correta depende da origem dos dados de entrada ou que produzem dados para ser usados em outro lugar (em geral, módulos de controle, entre outros, não seriam contados como parte de S_2)
- S_3 = número de módulos cuja função correta depende de processamento anterior
- S_4 = número de itens de base de dados (inclui objetos dados e todos os atributos que definem objetos)
- S_5 = número total de itens únicos de base de dados
- S_6 = número de segmentos de base de dados (diferentes registros ou objetos individuais)
- S_7 = número de módulos com uma única entrada e saída (processamento de exceção não é considerado múltipla saída)

“A medição pode ser vista como um desvio. Esse desvio é necessário porque os humanos, na sua maioria, não são capazes de tomar decisões claras e objetivas [sem um suporte quantitativo].”

Horst Zuse

Uma vez determinados os valores S_1 até S_7 para um programa de computador, podem ser calculados os seguintes valores intermediários:

Estrutura de programa: D_1 , em que D_1 é definido da seguinte maneira: Se o projeto da arquitetura foi desenvolvido por meio de um método distinto (por exemplo, projeto orientado a fluxo de dados ou projeto orientado a objeto), então $D_1 = 1$, caso contrário $D_1 = 0$.

Independência de módulo: $D_2 = 1 - \frac{S_2}{S_1}$

Módulos não dependentes de processamento anterior: $D_3 = 1 - \frac{S_3}{S_1}$

Tamanho da base de dados: $D_4 = 1 - \frac{S_5}{S_4}$

Divisão da base de dados: $D_5 = 1 - \frac{S_6}{S_4}$

Característica de entrada/saída do módulo: $D_6 = 1 - \frac{S_7}{S_1}$

Com os valores intermediários determinados, o DSQI é computado da seguinte maneira:

$$DSQI = \sum w_i D_i$$

em que $i = 1$ a 6, w_i é o peso relativo da importância de cada um dos valores intermediários e $\sum w_i = 1$ (se todos os D_i tiverem o mesmo peso, então $w_i = 0,167$).

O valor de DSQI para projetos passados pode ser determinado e comparado com um projeto que esteja atualmente em desenvolvimento. Se o DSQI for significativamente mais baixo do que a média, é aconselhável executar mais trabalho de projeto e revisão. De modo semelhante, se grandes alterações deverão ser feitas em um projeto, o efeito daquelas alterações sobre o DSQI pode ser calculado.

23.3.2 Métricas para projeto orientado a objeto

Há muita coisa sobre projeto orientado a objeto que é subjetivo – um projetista experiente “sabe” como caracterizar um sistema orientado a objeto de maneira que implemente efetivamente os requisitos do cliente. Mas, à medida que um modelo de projeto orientado a objeto cresce em tamanho e complexidade, uma visão mais objetiva das características do projeto pode favorecer tanto o projetista experiente (que obtém uma visão adicional) quanto o novato (que obtém uma indicação da qualidade que de outra forma não estaria disponível).

Em um tratamento detalhado de métricas de software para sistemas orientados a objeto, Whitmire [Whi97] descreve nove características distintas e mensuráveis de um projeto orientado a objeto:

 **Que características podem ser medidas quando avaliamos um projeto orientado a objeto?**

“Muitas das decisões para as quais eu tinha que me basear no folclore e em mitos podem agora ser tomadas por meio de dados quantitativos.”

Scott Whitmire

Tamanho. É definido em termos de quatro visualizações: população, volume, comprimento e funcionalidade. *População* é medida tomando-se uma contagem estática das entidades orientadas a objeto, como classes ou operações. Medidas de *volume* são idênticas às medidas de população, mas são coletadas dinamicamente – em determinado instante do tempo. *Comprimento* é a medida de uma cadeia de elementos de projeto interconectados (por exemplo, a extensão de uma árvore de herança é uma medida do comprimento). As métricas de *funcionalidade* proporcionam uma indicação indireta do valor fornecido ao cliente por uma aplicação orientada a objeto.

Complexidade. Assim como o tamanho, há muitas visões diferentes da complexidade do software [Zus97]. Whitmire vê a complexidade em termos de características estruturais examinando como as classes de um projeto orientado a objeto se inter-relacionam umas com as outras.

Acoplamento. As conexões físicas entre elementos do projeto orientado a objeto (por exemplo, o número de colaborações entre classes ou o número de mensagens passadas entre objetos) representam o acoplamento em um sistema orientado a objeto.

Suficiência. Whitmire define *suficiência* como “o grau com o qual uma abstração possui as características requeridas para ela, ou o grau com o qual um componente de projeto possui características em sua abstração, do ponto de vista da aplicação corrente”. Em outras palavras, perguntamos: “Que propriedades essa abstração (classe) precisa ter para ser útil para mim?” [Whi97]. Essencialmente, um componente de projeto (por exemplo, uma classe) é *suficiente* se refletir totalmente todas as propriedades do objeto de domínio da aplicação que está modelando – isto é, que a abstração (classe) possui as características necessárias para ele.

Totalidade. A única diferença entre totalidade e suficiência é “o conjunto de características em relação às quais comparamos a abstração ou o componente de projeto” [Whi97]. A *suficiência* compara a abstração do ponto de vista da aplicação corrente. A *totalidade* considera múltiplos pontos de vista, formulando a pergunta: “Que propriedades são necessárias para representar completamente o objeto de domínio do problema?”. Em virtude do critério da totalidade considerar diferentes pontos de vista, ele tem uma implicação indireta no grau com o qual a abstração ou o componente de projeto pode ser reutilizado.

Coesão. Assim como seu correspondente no software convencional, um componente orientado a objeto deverá ser projetado de maneira que tenha todas as operações funcionando em conjunto para atingir um objetivo único e bem definido. A coerência de uma classe é determinada examinando-se o grau segundo o qual “o conjunto de propriedades que ela possui faz parte do domínio do problema ou projeto” [Whi97].

Originalidade. Uma característica similar à simplicidade, originalidade (aplicada tanto a operações quanto a classes) é o grau segundo o qual uma operação é atômica – isto é, a operação não pode ser construída por meio de uma sequência de outras operações contidas dentro de uma classe. Uma classe que apresenta um alto grau de originalidade encapsula apenas operações primitivas.

Similaridade. O grau segundo o qual duas ou mais classes são similares em termos de sua estrutura, função, comportamento ou finalidade é indicado por essa medição.

Volatilidade. Conforme já afirmamos muitas vezes neste livro, podem ocorrer alterações de projeto quando os requisitos são modificados ou quando acontecem modificações em outras partes de um aplicativo, resultando em adaptação obrigatória do componente de projeto em questão. A volatilidade de um componente orientado a objeto mede a possibilidade de que uma alteração venha a ocorrer.

Na realidade, as métricas de produto para sistemas orientados a objeto podem ser aplicadas não só ao modelo de projeto, mas também ao modelo de requisitos. Nas próximas seções, discutiremos as métricas que fornecem indicação da qualidade em nível de classe orientada a

objeto e em nível de operação. Além disso, serão exploradas também métricas aplicáveis a gerenciamento de projeto e teste.

23.3.3 Métricas orientadas a classe — o conjunto de métricas CK

A classe é a unidade fundamental de um sistema orientado a objeto. Portanto, medidas e métricas para uma classe individual, para a hierarquia da classe e colaborações entre classes serão valiosas quando tivermos de avaliar qualidade de projeto orientado a objeto. Uma classe encapsula dados e a função manipula os dados. Com frequência é o “pai” das subclasses (às vezes chamadas de filhas) que herda seus atributos e operações. Muitas vezes elas colaboram com outras classes. Cada uma dessas características pode ser usada como base para a medida.⁷

Chidamber e Kemerer propuseram um dos conjuntos mais amplamente conhecidos de métricas de software orientado a objeto [Chi94]. Também chamado de conjunto de métricas CK (*CK metrics suite*), os autores propuseram seis métricas de projeto baseado em classe para sistemas orientados a objeto.⁸

Métodos ponderados por classe (*weighted methods per class – WMC*). Suponha que n métodos de complexidade $c_1, c_2, \dots, c_n$ são definidos para uma classe C . A métrica específica de complexidade escolhida (por exemplo, complexidade ciclomática) deverá ser normalizada de maneira que a complexidade nominal para um método assuma o valor 1.0.

$$WMC = \sum c_i$$

para $i = 1$ a n . O número de métodos e sua complexidade são indicadores razoáveis do trabalho necessário para implementar e testar uma classe. Além disso, quanto maior for o número de métodos, mais complexa será a árvore de herança (todas as subclasses herdam os métodos de seus pais). Por fim, conforme o número de métodos cresce para uma dada classe, ela tende a se tornar cada vez mais específica de aplicação, limitando assim sua potencial reutilização. Por todas essas razões, o WMC deverá ser mantido o mais baixo possível.

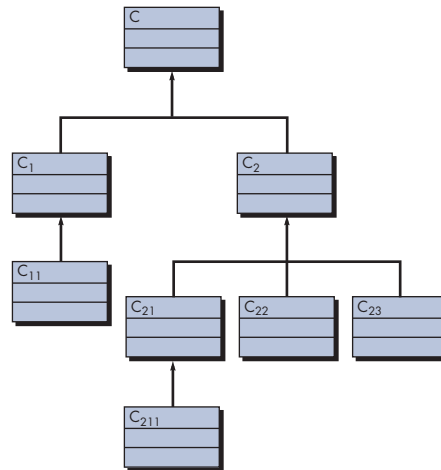
Embora pudesse parecer relativamente fácil desenvolver uma contagem para o número de métodos em uma classe, o problema é na realidade mais complexo do que parece. Deverá ser desenvolvida uma abordagem consistente de contagem [Chu95].

Extensão da árvore de herança (*depth of the inheritance tree – DIT*). Essa métrica é “o comprimento máximo desde o nó até a raiz da árvore” [Chi94]. De acordo com a Figura 23.5, o valor da DIT para a hierarquia de classe mostrada é 4. À medida que a DIT cresce, é possível que classes de nível inferior herdem muitos métodos. Isso causa dificuldades potenciais quando se tenta prever o comportamento de uma classe. Uma hierarquia de classe profunda (com DIT grande) também leva a uma complexidade maior no projeto. No lado positivo, grandes valores DIT implicam que muitos métodos podem ser reutilizados.

Número de filhas (*number of children – NOC*). As subclasses que estão imediatamente subordinadas a uma classe na hierarquia de classe são chamadas de suas filhas. De acordo com a Figura 23.5, a classe C_2 tem três filhas – as subclasses C_{21} , C_{22} e C_{23} . Conforme cresce o número de filhas, a reutilização aumenta, mas também à medida que o NOC aumenta, a abstração representada pela classe pai pode ser diluída se algumas das filhas não forem membros apropriados da classe pai. Conforme o NOC aumenta, a quantidade de teste (necessário para exercitar cada filha em seu contexto operacional) também aumentará.

⁷ Deve-se notar que a validade de algumas das métricas abordadas neste capítulo são atualmente discutidas na literatura técnica. Aqueles que defendem a teoria de medição demandam um grau de formalismo que algumas métricas orientadas a objeto não fornecem. No entanto, é razoável afirmar que as métricas citadas proporcionam informações úteis para o engenheiro de software.

⁸ Chidamber, Darcy e Kemerer usam o termo *métodos* em vez de *operações*. O uso desse termo é citado nesta seção.

FIGURA 23.5**Uma hierarquia de classe**

Os conceitos de acoplamento e coesão se aplicam ao software convencional e ao software orientado a objeto. Mantenha o acoplamento de classes baixo e a coesão de operação alta.

Acoplamento entre objetos de classes (coupling between object classes – CBO). O modelo CRC (Capítulo 6) pode ser usado para determinar o valor para o CBO. Essencialmente, CBO é o número de colaborações listadas para uma classe em seu cartão de indexação CRC.⁹ À medida que o CBO aumenta, é possível que a reutilização de uma classe diminua. Altos valores de CBO também complicam modificações e o teste resultante dessas modificações. Em geral, os valores de CBO para cada classe deverão ser mantidos o mais baixos possível; é consistente com a diretriz geral de reduzir o acoplamento em software convencional.

Resposta para uma classe (response for a class – RFC). O conjunto de respostas de uma classe é “um conjunto de métodos que podem potencialmente ser executados em resposta a uma mensagem recebida por um objeto daquela classe” [Chi94]. RFC é o número de métodos no conjunto de respostas. Conforme a RFC aumenta, o trabalho necessário para o teste também aumenta porque a sequência de testes (Capítulo 19) cresce. Segue também que, à medida que a RFC aumenta, a complexidade geral de projeto da classe aumenta.

Falta de coesão em métodos (lack of cohesion in methods – LCOM). Cada método dentro de uma classe **C** acessa um ou mais atributos (também chamados de variáveis de instância). LCOM é o número de métodos que acessam um ou mais dos mesmos atributos.¹⁰ Se nenhum método acessa os mesmos atributos, LCOM = 0. Para ilustrar o caso em que LCOM ≠ 0, considere uma classe com seis métodos. Quatro dos métodos têm um ou mais atributos em comum (eles acessam atributos comuns). Portanto, LCOM = 4. Se LCOM for alto, métodos podem ser acoplados uns aos outros via atributos. Isso aumenta a complexidade do projeto de classe. Embora haja casos em que um valor alto de LCOM seja justificável, é desejável manter a coesão alta; isto é, manter LCOM baixo.¹¹

⁹ Se os cartões de indexação CRC são desenvolvidos manualmente, a totalidade e a consistência devem ser avaliadas para que o CBO possa ser determinado de forma confiável.

¹⁰ A definição formal é um pouco mais complexa. Veja os detalhes em [Chi94].

¹¹ A métrica LCOM fornece informações úteis em algumas situações, mas pode ser confusa em outras. Por exemplo, mantendo-se o acoplamento encapsulado (dentro de uma classe), aumenta a coesão do sistema como um todo. Portanto, pelo menos em um sentido importante, LCOM mais alto realmente sugere que uma classe possa ter coesão mais alta, não mais baixa.

CASA SEGURA



Aplicando métricas CK

Cena: Sala do Vinod.

Atores: Vinod, Jamie, Shakira e Ed – membros da equipe de engenharia de software do CasaSegura que continuam a trabalhar no projeto em nível de componente e projeto de caso de teste.

Conversa:

Vinod: Vocês tiveram uma chance de ler a descrição sobre o conjunto de métricas CK que eu enviei na quarta-feira e fazer aquelas medidas?

Shakira: Não foi muito complicado. Utilizei minha classe UML e diagramas de sequência, como você sugeriu, e fiz contagens aproximadas de DIT, RFC e LCOM. Não consegui encontrar o modelo CRC, então não fiz contagem de CBO.

Jamie (rindo): Você não poderia encontrar o modelo CRC porque ele estava comigo.

Shakira: É isso que eu adoro nesta equipe, uma excelente comunicação.

Vinod: Fiz minhas contagens... Vocês desenvolveram valores para as métricas CK? [Jamie e Ed acenam afirmativamente.]

Jamie: Como eu tinha os cartões CRC, dei uma olhada no CBO e ele parecia bastante uniforme na maioria das classes. Havia apenas uma exceção.

Ed: Há algumas classes em que o RFC é muito alto, comparado com as médias... Talvez devêssemos dar um jeito de simplificá-los.

Jamie: Talvez sim, talvez não. Ainda estou preocupado com o prazo, e não quero corrigir coisas que não tenham realmente erros.

Vinod: Concorro com isso. Talvez vocês deveriam dar uma olhada nas classes que apresentam números ruins em pelo menos duas ou mais das métricas CK.

Shakira (observando a lista de classes de Ed com alto RFC): Olhe, veja esta classe, com LCOM e RFC ambos altos.

Vinod: Penso que sim... Será difícil de implementar devido à complexidade e difícil de testar pela mesma razão. Provavelmente vale a pena projetar duas classes separadas para conseguir o mesmo comportamento.

Jamie: Você acha que modificando economizaremos tempo?

Vinod: No longo prazo, sim.

“Analisar o software orientado a objeto para avaliar sua qualidade está se tornando cada vez mais importante na medida em que o paradigma orientado a objeto continua a crescer em popularidade.”

Rachel Harrison
et al.

23.3.4 Métricas orientadas a classe – o conjunto de métricas MOOD

Harrison, Counsell e Nithi [Har98b] propõem um conjunto de métricas para projeto orientado a objeto que proporciona indicadores quantitativos para características de projeto orientado a objeto. Veja a seguir exemplos das métricas MOOD.

Fator de herança de método (method inheritance factor – MIF). O grau segundo o qual a arquitetura de classe de um sistema orientado a objeto faz uso de herança tanto para métodos (operações) quanto para atributos. É definido como

$$MIF = \frac{\sum M_i(C_i)}{\sum M_a(C_i)}$$

em que a somatória ocorre com $i = 1$ até TC. TC é definido como o número total de classes na arquitetura, C_i é a classe dentro da arquitetura e

$$M_a(C_i) = M_d(C_i) + M_i(C_i)$$

em que

$M_d(C_i)$ = número de métodos que podem ser invocados em associação com C_i

$M_a(C_i)$ = número de métodos declarados na classe C_i

$M_i(C_i)$ = número de métodos herdados (e não cancelados) em C_i

O valor de MIF [o fator de herança de atributo (AIF) é definido de maneira análoga] fornece uma indicação do impacto da herança sobre o software orientado a objeto.

Fator de acoplamento (coupling factor – CF). Anteriormente, neste capítulo, observou-se que o acoplamento é uma indicação das conexões entre elementos do projeto orientado a objeto. O conjunto de métricas MOOD define o acoplamento da seguinte maneira:

$$CF = \sum_i \sum_j is_client \frac{(C_i, C_j)}{T_c^2 - T_c}$$

em que a somatória de $i = 1$ a T_c e $j = 1$ a T_c . A função

$$is_client = 1, \text{ se, e somente se, existir uma relação entre a classe-cliente } C_c \text{ e a classe-servidora } C_s, \text{ e } C_c \neq C_s \\ = 0, \text{ em caso contrário}$$

Embora muitos fatores afetem a complexidade, entendimento e sustentabilidade do software, é razoável concluir que, conforme o fator CF aumenta, a complexidade do software orientado a objeto também aumenta e o entendimento, a sustentabilidade e o potencial de reutilização podem ficar prejudicados.

Harrison e seus colegas [Har98b] apresentam uma análise detalhada do MIF e CF com suas métricas e examinam sua validade para uso na avaliação da qualidade do projeto.

23.3.5 Métricas orientadas a objeto propostas por Lorenz e Kidd

Em seu livro sobre métricas orientadas a objeto, Lorenz e Kidd [Lor94] dividem as métricas baseadas em classe em quatro categorias principais, cada uma delas ocupando uma posição no projeto em nível de componente: tamanho, herança, aspectos internos e externos. As métricas orientadas a tamanho para uma classe de projeto orientado a objeto focalizam contagens de atributos e operações para uma classe individual e valores médios para o sistema orientado a objeto como um todo. Métricas baseadas em herança focalizam a maneira pela qual as operações são reutilizadas através da hierarquia de classe. Métricas para aspectos internos de classe procuram a coesão (Seção 23.3.3) e questões orientadas a código, e as métricas de aspectos externos examinam acoplamento e reutilização. Um exemplo das métricas propostas por Lorenz e Kidd é:

Tamanho de classe (class size – CS). O tamanho global de uma classe pode ser determinado por meio das seguintes medidas:

- número total de operações (operações de instâncias herdadas e privadas) que são encapsuladas dentro da classe
- número de atributos (atributos de instâncias herdadas e privadas) que são encapsulados pela classe

A métrica WMC proposta por Chidamber e Kemerer (Seção 23.3.3) é também uma medida ponderada do tamanho da classe. Conforme observamos, valores altos de CS indicam que uma classe pode ter muita responsabilidade. Isso reduzirá a reutilização da classe e complicará a implementação e o teste. Em geral, operações e atributos, herdados ou públicos, deverão ser ponderados mais intensamente na determinação do tamanho da classe [Lor94]. Operações privadas e atributos privados permitem especialização e são mais localizados no projeto. Podem ser calculados também valores médios para o número de atributos e operações de classe. Quanto mais baixos os valores médios para o tamanho, maior probabilidade terão as classes dentro do sistema de ser reutilizadas mais amplamente.

23.3.6 Métricas de projeto em nível de componente

As métricas de projeto em nível de componente para componentes convencionais de software focalizam características internas de um componente de software e incluem medidas dos “três Cs” – coesão de módulo (*module cohesion*), acoplamento (*coupling*) e complexidade (*complexity*). Essas medidas podem ajudá-lo a avaliar a qualidade de um projeto em nível de componente.



Durante a revisão do modelo de análise, os cartões de indexação CRC fornecerão uma indicação razoável dos valores esperados para CS. Se você encontrar uma classe com um alto número de responsabilidades, pense em dividi-la.

PONTO-CHAVE

É possível computar medidas da independência funcional – acoplamento e coesão – de um componente e usá-las para avaliar a qualidade de um projeto.

Métricas de projeto em nível de componente podem ser aplicadas uma vez desenvolvido o projeto procedural; elas são uma “caixa de vidro” pelo fato de requererem conhecimento do funcionamento interno do módulo que está sendo considerado. Como alternativa, elas podem ser adiadas até que o código-fonte esteja disponível.

Métricas de coesão. Definidas por Bieman e Ott [Bie94] como uma coleção de métricas que fornecem a indicação da coerência (Capítulo 8) de um módulo. As métricas são caracterizadas em termos de cinco conceitos e medidas:

- *Fatia de dados (data slice).* Em poucas palavras, é a busca retroativa através de um módulo à procura por valores de dados que afetam a localização do módulo na qual essa busca começou. Devemos observar que podem ser definidas tanto as fatias de programa (que focalizam instruções e condições) como as fatias de dados.
- *Fichas de dados (data tokens).* As variáveis definidas para um módulo podem ser consideradas fichas de dados para o módulo.
- *Fichas aglutinantes (glue tokens).* Conjunto de fichas de dados que residem em uma ou mais fatias de dados.
- *Fichas superaglutinantes (superglue tokens).* Fichas de dados comuns a todas as fatias de dados em um módulo.
- *Aglutinação (stickiness).* A aglutinação relativa de uma ficha aglutinante é diretamente proporcional ao número de fatias de dados que ele une.

Bieman e Ott desenvolvem métricas para *coesão funcional forte (strong functional cohesion - SFC)*, *coesão funcional fraca (weak functional cohesion - WFC)* e adesividade (*adhesiveness*) - o grau relativo em que fichas aglutinantes juntam fatias de dados entre si. Para uma discussão detalhada das métricas de Bieman e Ott, é aconselhável consultar os próprios autores [Bie94].

Métricas de acoplamento (coupling metrics). O acoplamento de módulo fornece uma indicação da “conexão” de um módulo com outros módulos, dados globais e o ambiente externo. No Capítulo 9, o acoplamento foi discutido em termos qualitativos.

Dhama [Dha95] propôs uma métrica para acoplamento de módulo, que abrange acoplamento de dados, de fluxo de controle, global e ambiental. As medidas necessárias para computar o acoplamento de módulo são definidas em relação a cada um dos três tipos de acoplamento citados anteriormente.

Para acoplamento de dados e de fluxo de controle,

$$\begin{aligned}d_i &= \text{número de parâmetros de dados de entrada} \\c_i &= \text{número de parâmetros de controle de entrada} \\d_o &= \text{número de parâmetros de dados de saída} \\c_o &= \text{número de parâmetros de controle de saída}\end{aligned}$$

Para acoplamento global,

$$\begin{aligned}g_d &= \text{número de variáveis globais usadas como dados} \\g_c &= \text{número de variáveis globais usadas como controle}\end{aligned}$$

Para acoplamento ambiental,

$$\begin{aligned}w &= \text{número de módulos chamados (fan-out)} \\r &= \text{número de módulos que chamam o módulo em consideração (fan-in)}\end{aligned}$$

Usando essas medidas, um indicador de acoplamento de módulo m_c é definido da seguinte maneira:

$$m_c = \frac{k}{M}$$

em que k é uma constante de proporcionalidade e

$$M = d_i + (a \times c_i) + d_o + (b \times c_o) + g_d + (c \times g_c) + w + r$$

Valores para k , a , b e c devem ser obtidos empiricamente.

À medida que o valor para mc aumenta, o acoplamento global do módulo diminui. Para que a métrica de acoplamento suba quando o grau de acoplamento aumenta, uma métrica de acoplamento revisada pode ser definida como

$$C = 1 - m_c$$

em que o grau de acoplamento aumenta quando o valor de M aumenta.

Métricas de complexidade (complexity metrics). Uma variedade de métricas de software pode ser computada para determinar a complexidade do fluxo de controle do programa. Muitas dessas métricas são baseadas no diagrama de fluxo. Um gráfico (Capítulo 18) é uma representação composta de nós e vínculos (também chamados de *edges*). Quando os vínculos estão direcionados, o diagrama de fluxo é um grafo direcionado.

McCabe e Watson [McC94] identificam uma série de usos importantes para as métricas de complexidade:

Métricas de complexidade podem ser usadas para prever informações críticas sobre a confiabilidade e sustentabilidade de sistemas de software com base em análise automática do código-fonte [ou informações sobre o projeto procedural]. Métricas de complexidade também proporcionam informações durante o projeto de software para ajudar a controlar a [atividade de projeto]. Durante os testes e a manutenção, elas fornecem informações detalhadas sobre os módulos de software para ajudar a apontar áreas de instabilidade potencial.

A métrica de complexidade mais usada (e debatida) para software é a ciclomática, desenvolvida por Thomas McCabe [McC76] e discutida em detalhes no Capítulo 18.

Zuse ([Zus90], [Zus97]) apresenta uma discussão enciclopédica de mais de 18 diferentes categorias de métricas de complexidade de software. O autor aborda as definições básicas para métricas em cada categoria (por exemplo, há um conjunto de variações na métrica de complexidade ciclomática) e então analisa e critica cada uma delas. O trabalho de Zuse é o mais abrangente publicado até agora.

23.3.7 Métricas orientadas a operação

Devido à classe ser a unidade dominante nos sistemas orientados a objeto, poucas métricas têm sido propostas para operações que residem dentro de uma classe. Churcher e Shepperd [Chu95] discutem isso quando afirmam: “Resultados de estudos recentes indicam que métodos tendem a ser pequenos, tanto em termos de número de instruções quanto em complexidade lógica [Wil93], sugerindo que a estrutura de conectividade de um sistema pode ser mais importante do que o conteúdo de módulos individuais”. No entanto, podem ser obtidas algumas informações examinando-se as características médias dos métodos (operações). Três métricas simples, propostas por Lorenz e Kidd [Lor94], são apropriadas:

Tamanho médio de operação (average operation size – OS_{avg}). O tamanho pode ser determinado contando-se o número de linhas de código ou o número de mensagens enviadas pela operação. Na medida em que aumenta o número de mensagens enviadas por uma única operação, é possível que as responsabilidades não tenham sido bem alocadas dentro da classe.

Complexidade de operação (operation complexity – OC). A complexidade de uma operação pode ser calculada usando-se quaisquer métricas de complexidade propostas para software convencional [Zus90]. Como as operações deverão ser limitadas a uma responsabilidade específica, o projetista deve se esforçar para manter a OC o mais baixo possível.

PONTO-CHAVE

A complexidade ciclomática é apenas uma dentre inúmeras métricas de complexidade.

Número médio de parâmetros por operação (*average number of parameters per operation* – NP_{avg}). Quanto maior o número de parâmetros de operação, mais complexa é a colaboração entre objetos. Em geral, NP_{avg} deverá ser mantida o mais baixo possível.

23.3.8 Métricas de projeto de interface de usuário

“Você pode aprender pelo menos um princípio de projeto de interface de usuário quando utiliza uma lavadora de louças. Se você colocar muitos objetos lá dentro, nada ficará muito limpo.”

Autor desconhecido

Embora haja uma literatura significativa sobre o projeto de interface humanos/computadores (Capítulo 11), têm sido publicadas poucas informações sobre métricas que poderiam fornecer informações sobre a qualidade e utilização da interface.

Sears [Sea93] sugere que a *adequação do layout* (*layout appropriateness* LA) é uma métrica de projeto valiosa para interfaces humanos/computadores. Uma GUI típica usa entidades de layout – ícones gráficos, texto, menus, janelas e itens do gênero – para ajudar o usuário na execução das tarefas. Para executar uma tarefa usando uma GUI, o usuário deve passar de uma entidade de layout para a próxima. A posição absoluta e relativa de cada entidade de layout, a frequência com a qual ela é usada e o “custo” da transição de uma entidade de layout para a próxima contribuem para a adequação da interface.

Um estudo das métricas para páginas da Web [Ivo01] indica que características simples dos elementos do layout podem também ter um impacto significativo sobre a qualidade aparente do projeto da GUI. O número de palavras, links, gráficos, cores e fontes (entre outras características) contidas em uma página da Web afetam a complexidade aparente e a qualidade dela.

É importante notar que a seleção de um projeto de GUI pode ser guiada com métricas como LA, mas o julgamento final deverá ser a entrada de usuário baseada em protótipos de GUI. Nielsen e Levy [Nie94] relatam que “há uma chance razoavelmente grande de sucesso se alguém escolher entre interface [projetos] baseada unicamente nas opiniões dos usuários. O desempenho médio da tarefa do usuário e sua satisfação subjetiva com uma GUI estão altamente correlacionados.”

23.4 MÉTRICAS DE PROJETO PARA WEBAPPS

Um bom conjunto de medidas e métricas para WebApps fornece respostas quantitativas para as seguintes questões:

- A interface de usuário promove a utilização?
- O aspecto estético da WebApp é apropriado para o domínio de aplicação e é agradável ao usuário?
- O conteúdo é projetado de forma a reunir o máximo de informações com o mínimo esforço?
- A navegação é eficiente e direta?
- A arquitetura da WebApp foi projetada para acomodar as metas e objetivos especiais de seus usuários, a estrutura de conteúdo e funcionalidade e o fluxo de navegação requerido para usar o sistema eficientemente?
- Os componentes são projetados de maneira que reduzam a complexidade de procedimento e melhorem a exatidão, confiabilidade e desempenho?



Muitas dessas métricas são aplicáveis a todas as interfaces de usuário e devem ser consideradas em conjunto com as apresentadas na Seção 23.3.8.

Atualmente, cada uma dessas questões pode ser resolvida apenas qualitativamente porque não existe um conjunto qualificado de métricas que forneceriam respostas quantitativas.

Nos próximos parágrafos, apresentamos uma amostra representativa das métricas de projeto para WebApp propostas na literatura. É importante notar que muitas dessas métricas ainda não foram validadas e deverão ser usadas com muito critério.

Métricas de interface. Para WebApps, podem ser consideradas as seguintes medidas de interface:

<i>Métrica sugerida</i>	<i>Descrição</i>
Adequação do layout	Veja a Seção 23.3.8.
Complexidade de layout	Número de regiões distintas ¹² definidas para uma interface
Complexidade da região de layout	Número médio de links distintos por região
Complexidade de reconhecimento	Número médio de itens distintos que o usuário deve examinar antes de tomar uma decisão de navegação ou de entrada de dados
Tempo de reconhecimento	Tempo médio (em segundos) que o usuário gasta para selecionar a ação apropriada para uma tarefa
Trabalho de digitação	Número médio de toques necessários para uma função específica
Esforço de click do mouse	Número médio de cliques do mouse por função
Complexidade de seleção	Número médio de links que podem ser selecionados por página
Tempo de aquisição de conteúdo	Número médio de palavras de texto por página da Web
Carga de memória	Número médio de itens de dados distintos que o usuário deve lembrar para atingir um objetivo específico

Métricas de estética (design gráfico). Por sua natureza, o projeto estético depende da avaliação qualitativa e não é geralmente favorável à medição e às métricas. No entanto, Ivory e seus colegas [Ivo01] propõem um conjunto de medidas que podem ser úteis para avaliar o impacto do projeto estético:

<i>Métrica Sugerida</i>	<i>Descrição</i>
Número de palavras	Número total de palavras que aparecem em uma página
Porcentagem de texto de corpo	Porcentagem de palavras que são texto de corpo <i>versus</i> texto de título (cabecinhos)
Texto de corpo destacado %	Parte do texto de corpo que é destacado (por exemplo, negrito, caixa alta)
Contagem de posicionamento de texto	Mudanças na posição do texto a partir do alinhamento à esquerda
Contagem de grupos de texto	Áreas de texto destacadas com cores, regiões com bordas, réguas ou listas
Contagem de link	Total de links em uma página
Tamanho de página	Total de bytes na página, bem como elementos, gráficos e folhas de estilo
Porcentagem gráfica	Porcentagem de bytes da página que são usados para gráficos (figuras)
Contagem gráfica	Total de figuras na página (não incluindo figuras especificadas em scripts, applets e objetos)
Contagem de cores	Total de cores usadas
Contagem de fonte	Total de fontes empregadas (face + size + bold + italic)

Métricas de conteúdo. Métricas nessa categoria focalizam a complexidade de conteúdo e clusters de objetos conteúdo que são organizados em páginas [Men01].

¹² Uma região distinta é uma área do layout de tela que executa um conjunto específico de funções relacionadas (por exemplo, uma barra de menu, uma tela gráfica estática, uma área de conteúdo, uma tela animada).

<i>Métrica Sugerida</i>	<i>Descrição</i>
Espera de página	Tempo médio necessário para que uma página seja baixada em diferentes velocidades de conexão
Complexidade da página	Número médio de diferentes tipos de mídia usada na página, não incluindo texto
Complexidade gráfica	Número médio de mídia gráfica por página
Complexidade de áudio	Número médio de mídias de áudio por página
Complexidade de vídeo	Número médio de mídia de vídeo por página.
Complexidade de animação	Número médio de animações por página
Complexidade de imagem escaneada	Número médio de imagens escaneadas por página

Métricas de navegação. Métricas nessa categoria tratam da complexidade do fluxo de navegação [Men01]. Em geral, são direcionadas apenas para aplicações Web estáticas, que não incluem links e páginas gerados dinamicamente.

<i>Métrica Sugerida</i>	<i>Descrição</i>
Complexidade de link de página	Número de links por página
Conectividade	Número total de links internos, não incluindo links gerados dinamicamente
Densidade de conectividade	Conectividade dividida por número de página

Com um subconjunto das métricas sugeridas, pode ser possível derivar relações empíricas que permitem a uma equipe de desenvolvimento de WebApp avaliar a qualidade técnica e prever o trabalho necessário com base nas estimativas projetadas da complexidade. Há ainda muito a ser feito nesta área.

FERRAMENTAS DO SOFTWARE



Métricas técnicas para WebApps

Objetivo: ajudar os engenheiros da Web a desenvolver métricas para WebApp com significado que forneçam informações sobre a qualidade global de um aplicativo.

Mecânica: o mecanismo das ferramentas é variado.

Ferramentas representativas:¹³ Netmechanic Tools, desenvolvida pela Netmechanic (www.netmechanic.com), é uma coleção de ferramentas que ajudam a melhorar o desempenho do site, focalizando assuntos específicos de implementação. NIST Web Metrics Testbed, desenvolvida pelo National Institute of Standards and Technology (zing.ncsl.nist.gov/WebTools/), abrange a seguinte coleção de ferramentas úteis disponíveis para download:

Web Static Analyzer Tool (WebSAT) – verifica o HTML da página da Web segundo diretrizes de utilização típica.

Web Category Analysis Tool (WebCAT) – permite ao engenheiro de utilização criar e executar uma análise de categoria Web.

Web Variable Instrumenter Program (WebVIP) – capacita um site a capturar um log de interação de usuário.

Framework for Logging Usability Data (FLUD) – implementa um formatador de arquivo e um analisador sintático (*parser*) para representação dos logs de interação do usuário.

VisVIP Tool – produz uma visualização 3D dos caminhos de navegação do usuário através de um site.

TreeDec – acrescenta auxílios de navegação às páginas de um site.

23.5 MÉTRICAS PARA CÓDIGO-FONTE

A teoria de Halstead da “ciência do software” [Hal77] propôs as primeiras “leis” analíticas para programas de computador.¹⁴ Halstead atribuiu leis quantitativas ao desenvolvimento de

¹³ As ferramentas aqui apresentadas não significam um aval, mas sim uma amostra dessa categoria. Na maioria dos casos, seus nomes são marcas registradas pelos respectivos desenvolvedores.

¹⁴ Deve-se observar que as “leis” de Halstead geraram controvérsias substanciais, e muitos acreditam que a teoria na qual se baseiam apresenta falhas. No entanto, executaram-se verificações experimentais para algumas linguagens de programação (por exemplo, [Fel89]).

"O cérebro humano segue uma série de regras mais rígidas [para desenvolver algoritmos] do que ele próprio poderia imaginar."

Maurice Halstead

software, usando um conjunto de medidas primitivas que podem ser obtidas depois que o código é gerado, ou estimadas quando o projeto estiver completo. As medidas são:

n_1 = número de operadores distintos que aparecem em um programa

n_2 = número de operandos distintos que aparecem em um programa

N_1 = número total de ocorrências de operador

N_2 = número total de ocorrências de operando

Halstead usa essas medidas primitivas para desenvolver expressões para o tamanho global do programa, volume mínimo potencial para um algoritmo, o volume atual (número de bits necessários para especificar um programa), nível do programa (medida da complexidade do software), nível de linguagem (constante para uma dada linguagem) e outras características como esforço de desenvolvimento, tempo de desenvolvimento e até um número projetado de falhas no software.

Halstead mostra que o tamanho N pode ser estimado da seguinte maneira:

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

e o volume do programa pode ser definido como:

$$V = N \log_2 (n_1 + n_2)$$

Devemos notar que V varia com a linguagem de programação e representa o volume de informações (em bits) necessário para especificar um programa.

Teoricamente, deve existir um volume mínimo para determinado algoritmo. Halstead define a razão de volume L como a razão entre o volume da forma mais compacta de um programa e o volume do programa real. Na realidade, L deve ser sempre menor do que 1. Em termos de medidas primitivas, a relação de volume pode ser expressa como

$$L = \frac{2}{n_1} \times \frac{n_2}{N_2}$$

O trabalho de Halstead é favorável à verificação experimental e muitas pesquisas já foram feitas para investigar a ciência do software. Uma discussão desse trabalho está além do escopo deste livro. Para mais informações, veja [Zus90], [Fen91] e [Zus97].



Operadores incluem todas as construções (constructs) de controle de fluxo, operações condicionais e matemáticas. Operandos abrangem todas as variáveis e constantes de programas.

23.6 MÉTRICAS PARA TESTE

PONTO-CHAVE

As métricas de teste se classificam em duas grandes categorias: (1) métricas que tentam prever o número provável de testes necessários em vários níveis de teste e (2) métricas que focalizam a abrangência do teste para determinado componente.

Embora muito se tenha escrito sobre métricas de software para teste (por exemplo, [Het93]), a maioria das métricas propostas focaliza o processo de teste, não as características técnicas dos próprios testes. Em geral, os testadores precisam se basear nas métricas de análise, projeto e código para guiá-los no projeto e execução dos casos de teste.

As métricas de projeto da arquitetura fornecem informações sobre a facilidade ou dificuldade associadas com o teste de integração (Seção 23.3) e a necessidade de software de teste especializado (por exemplo, pseudocontroladores (*drivers*) e pseudocontrolados (*stubs*)). A complexidade ciclomática (uma métrica de projeto em nível de componente) depende do teste de caminho básico, um método de projeto de caso de teste apresentado no Capítulo 18. Além disso, a complexidade ciclomática pode ser usada para escolher módulos como candidatos para testes de unidade extensivos. Módulos com alta complexidade ciclomática são mais propensos a apresentar erro do que os de complexidade menor. Por essa razão, você deve dispendar esforços acima da média para descobrir erros nesses módulos antes de serem integrados em um sistema.

23.6.1 Métricas de Halstead aplicadas ao teste

O trabalho de teste pode ser estimado por meio de métricas obtidas das medidas de Halstead (Seção 23.5). Usando as definições para volume V de programa e nível PL , o trabalho Halstead e pode ser calculado como

$$PL = \frac{1}{(n_1/2) \times (N_2/n_2)} \quad (23.2a)$$

$$e = \frac{V}{PL} \quad (23.2b)$$

A porcentagem de trabalho de teste global a ser alocado a um módulo k pode ser estimada usando a seguinte relação:

$$\text{Porcentagem de trabalho de teste } (k) = \frac{e(k)}{\sum e(i)} \quad (23.3)$$

em que $e(k)$ é calculado para o módulo k por meio das Equações (23.2), e a somatória no denominador da Equação (23.3) é a soma do trabalho Halstead por todos os módulos do sistema.

23.6.2 Métricas para teste orientado a objeto

As métricas de projeto orientado a objeto apresentadas na Seção 23.3 fornecem uma indicação da qualidade do projeto. Elas fornecem também uma indicação geral do trabalho de teste necessário para exercitar um sistema orientado a objeto. Binder [Bin94b] sugere um conjunto amplo de métricas de projeto que têm influência direta sobre a “testabilidade” de um sistema orientado a objeto. As métricas consideram aspectos do encapsulamento e herança.

Falta de coesão em métodos (*lack of cohesion in methods – LCOM*).¹⁵ Quanto mais alto o valor de LCOM, mais estados precisam ser testados para garantir que os métodos não gerem efeitos colaterais.

Porcentagem pública e protegida (*percent public and protected – PAP*). Atributos públicos são herdados de outras classes e, portanto, são visíveis àquelas classes. Atributos protegidos são acessíveis a métodos em subclasses. Essa métrica indica a porcentagem de atributos de classe públicos ou protegidos. Altos valores para PAP aumentam a probabilidade de efeitos colaterais entre classes porque atributos públicos e protegidos levam a um alto potencial de acoplamento.¹⁶ Os testes devem ser projetados para garantir que esses efeitos colaterais sejam descobertos.

Acesso público a membros de dados (*public access to data members – PAD*). Essa métrica indica o número de classes (ou métodos) que podem acessar atributos de uma outra classe, uma violação do encapsulamento. Altos valores de PAD levam a efeitos colaterais em potencial entre classes. Os testes devem ser projetados de forma que garantam que esses efeitos colaterais sejam descobertos.

Número de classes-raiz (*number of root classes – NOR*). Essa métrica é uma contagem das hierarquias distintas de classe descritas no modelo de projeto. Devem ser desenvolvidos conjuntos de testes para cada classe-raiz e hierarquia de classe correspondente. À medida que o NOR aumenta, o trabalho de teste também aumenta.



O teste orientado a objeto pode ser muito complexo. As métricas podem ajudá-lo a direcionar os recursos de teste para threads, cenários e pacotes de classes que são “suspeitos” com base nas características medidas. Use-as.

¹⁵ Veja na Seção 23.3.3 uma descrição de LCOM.

¹⁶ Algumas pessoas fazem projetos nos quais nenhum dos atributos é público ou privado, ou seja, $PAP = 0$. Isso implica que todos os atributos devem ser acessados em outras classes via métodos.

Fan-in (FIN). Quando usado no contexto orientado a objeto, fan-in na hierarquia de herança é uma indicação de herança múltipla. $FIN > 1$ indica que uma classe herda seus atributos e operações de mais de uma classe-raiz. $FIN > 1$ deve ser evitado sempre que possível.

Número de filhos (number of children – NOC) e altura da árvore de herança (depth of the inheritance tree – DIT).¹⁷ Conforme mencionamos no Capítulo 19, métodos de superclasse terão de ser retestados para cada subclasse.

23.7 MÉTRICAS PARA MANUTENÇÃO

Todas as métricas de software introduzidas neste capítulo podem ser usadas para o desenvolvimento de novo software e para manutenção do software existente. No entanto, existem também métricas projetadas explicitamente para atividades de manutenção.

A norma IEEE Std. 982.1-1988 [IEE93] sugere um *índice de maturidade de software* (software maturity index – SMI) que fornece uma indicação da estabilidade de um produto (com base nas alterações que ocorrem para cada versão do produto). São determinadas as seguintes informações:

M_T = número de módulos na versão atual

F_c = número de módulos na versão atual que foram alterados

F_a = número de módulos na versão atual que foram acrescentados

F_d = número de módulos da versão anterior que foram excluídos na versão atual

O índice de maturidade de software é calculado da seguinte maneira:

$$SMI = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

À medida que o SMI se aproxima de 1.0, o produto começa a estabilizar. O SMI pode ser usado também como uma métrica para planejamento de atividades de manutenção de software. O tempo médio para produzir uma versão de um software pode ser correlacionado com o SMI, e podem ser desenvolvidos modelos empíricos para o trabalho de manutenção.

FERRAMENTAS DO SOFTWARE



Métricas de produto

Objetivo: ajudar os engenheiros de software no desenvolvimento de métricas com significado que avaliam os produtos gerados durante o projeto de análise e modelagem, geração de código-fonte e teste.

Mecânica: as ferramentas desta categoria abrangem um grande conjunto de métricas e são implementadas como uma aplicação *stand-alone* ou (mais comumente) como funcionalidade que existe dentro das ferramentas para análise e projeto, codificação ou teste. Em muitos casos, a ferramenta métrica analisa uma representação do software (por exemplo, um modelo UML ou código-fonte) e desenvolve uma ou mais métricas como resultado.

Ferramentas representativas:¹⁸ *Krakatau Metrics*, desenvolvida pela Power Software (www.powersoftware.com/products), calcula complexidade, Halstead e métricas relacionadas a C/C++ e Java.

Metrics4C – desenvolvida pela +1 Software Engineering (www.plus-one.com/Metrics4C_fact_sheet.html), calcula uma variedade de métricas de arquitetura, projeto e orientadas a código, bem como métricas orientadas a projeto.

Rational Rose, distribuída pela IBM (www-304.ibm.com/jct03001c/software/awdtools/developer/rose/), é um conjunto de ferramentas para modelagem UML que incorpora uma série de características de análise de métricas.

RSM, desenvolvida pela M-Squared Technologies (msquaredtechnologies.com/m2rsm/index.html), calcula uma grande variedade de métricas orientadas a código para C, C++ e Java.

Understand, desenvolvida pela Scientific Toolworks, Inc. (www.scitools.com), calcula métricas orientadas a código para uma variedade de linguagens de programação.

¹⁷ Veja na Seção 23.3.3 uma descrição de NOC e DIT.

¹⁸ As ferramentas aqui apresentadas não significam um aval, mas sim uma amostra dessa categoria. Na maioria dos casos, seus nomes são marcas registradas pelos respectivos desenvolvedores.

23.8 RESUMO

As métricas de software proporcionam uma maneira quantitativa de avaliar a qualidade dos atributos internos do produto, possibilitando avaliar a qualidade antes que ele seja criado. As métricas proporcionam as informações necessárias para criar requisitos eficazes e modelos de projeto, código sólido e testes completos.

Para ser útil na prática, uma métrica de software deve ser simples e computável, persuasiva, consistente e objetiva. Ela deve ser independente da linguagem de programação e fornecer um retorno eficaz.

Métricas para o modelo de requisitos focalizam função, dados e comportamento – os três componentes do modelo. Métricas para projeto consideram aspectos de arquitetura, projeto em nível de componente e projeto de interface. Métricas de projeto da arquitetura consideram os aspectos estruturais do modelo de projeto. Métricas de projeto em nível de componente fornecem indicação da qualidade do módulo, estabelecendo medidas indiretas para coesão, acoplamento e complexidade. Métricas de projeto de interface de usuário fornecem indicação da facilidade com que uma GUI pode ser usada. Métricas para WebApp consideram aspectos da interface de usuário, bem como a estética da WebApp, conteúdo e navegação.

Métricas para sistemas orientados a objeto concentram-se em medidas que podem ser aplicada às características da classe e do projeto – localização, encapsulamento, ocultamento de informações, herança e técnicas de abstração de objeto – que tornam a classe única. O conjunto de métricas CK define seis métricas de software orientado à classe que focalizam a classe e a hierarquia de classe. O conjunto de métricas também desenvolve métricas para avaliar as colaborações entre classes e a coesão de métodos que residem em uma classe. Em um nível orientado a classe, o conjunto de métricas CK pode ser ampliado com métricas propostas por Lorenz e Kidd e o conjunto de métricas MOOD.

Halstead apresenta um conjunto fascinante de métricas em nível de código-fonte. Usando uma série de operadores e operandos presentes no código, a ciência do software fornece uma variedade de métricas que podem ser usadas para avaliar a qualidade do programa.

Poucas métricas de produto têm sido propostas para uso direto em teste e manutenção de software. No entanto, muitas outras podem ser empregadas para orientar o processo de teste e como mecanismo para avaliar a sustentabilidade de um programa de computador. Uma grande variedade de métricas orientadas a objeto tem sido proposta para avaliar a testabilidade de um sistema orientado a objeto.

PROBLEMAS E PONTOS A PONDERAR

23.1. A teoria de medidas é um tópico avançado que tem grande influência sobre as métricas de software. Usando [Zus97], [Fen91], [Zus90] ou fontes baseadas na Web, escreva um pequeno artigo que destaque os principais princípios da teoria das medidas. Projeto individual: desenvolva uma apresentação sobre o assunto e apresente para a sua classe.

23.2. Por que não é possível desenvolver uma métrica única e totalmente abrangente para complexidade de programa ou qualidade de programa? Tente criar uma medida ou métrica utilizada na prática que contrarie os atributos de métricas eficazes de software definidas na Seção 23.1.5.

23.3. Um sistema tem 12 entradas externas, 24 saídas externas, responde a 30 diferentes consultas externas, gerencia 4 arquivos de lógica interna e estabelece interface com 6 diferentes sistemas legados (6 EIFs). Todos esses dados são de complexidade média e o sistema global é relativamente simples. Calcule FP para o sistema.

23.4. O software para o Sistema X tem 24 requisitos funcionais e 14 requisitos não funcionais. Qual a peculiaridade dos requisitos? A totalidade?

23.5. Um grande sistema de informações tem 1.140 módulos. Há 96 módulos que executam funções de controle e coordenação e 490 módulos cuja função depende de processamento

anterior. O sistema processa aproximadamente 220 objetos de dados tendo cada um deles em média três atributos. Há 140 itens distintos de base de dados e 90 diferentes segmentos de base de dados. Por fim, 600 módulos têm pontos distintos de entrada e saída. Calcule o DSQI para esse sistema.

23.6. Uma classe **X** tem 12 operações. A complexidade ciclomática foi computada para todas as operações no sistema orientado a objeto, e o valor médio da complexidade de módulo é 4. Para a classe **X**, a complexidade para as operações 1 a 12 é 5, 4, 3, 3, 6, 8, 2, 2, 5, 5, 4, 4, respectivamente. Calcule os métodos ponderados por classe.

23.7. Desenvolva uma ferramenta de software que calculará a complexidade ciclomática para um módulo de linguagem de programação. Você pode escolher a linguagem.

23.8. Desenvolva uma pequena ferramenta de software que executará uma análise Halstead sobre código-fonte de linguagem de programação de sua escolha.

23.9. Um sistema legado tem 940 módulos. A última versão requeria que 90 desses módulos fossem alterados. Além disso, 40 novos módulos foram acrescentados e 12 módulos antigos foram removidos. Calcule o índice de maturidade do software para o sistema.

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

Há uma quantidade surpreendentemente grande de livros dedicados às métricas de software, embora a maioria deles focalize métricas de processo e projeto não incluindo métricas de produto. Lanza e seus colegas (*Object-Oriented Metrics in Practice*, Springer, 2006) discutem as métricas orientadas a objeto e seu uso para analisar a qualidade de um projeto. Genero (*Metrics for Software Conceptual Models*, Imperial College Press, 2005) e Ejiogu (*Software Metrics*, Book-Surge Publishing, 2005) apresentam uma grande variedade de métricas técnicas para casos de uso, modelos UML e outras representações de modelagem. Hutcheson (*Software Testing Fundamentals: Methods and Metrics*, Wiley, 2003) aborda um conjunto de métricas para teste. Kan (*Metrics and Models in Software Quality Engineering*, Addison-Wesley, 2d ed., 2002), Fenton e Pfleeger (*Software Metrics: A Rigorous and Practical Approach*, Brooks-Cole Publishing, 1998) e Zuse [Zus97] escreveram tratamentos completos sobre métricas de produto.

Os livros de Card e Glass [Car90], Zuse [Zus90], Fenton [Fen91], Ejiogu [Eji91], Moeller e Paulish (*Software Metrics*, Chapman e Hall, 1993), e Hetzel [Het93] tratam de métricas de produto com algum detalhe. Oman e Pfleeger (*Applying Software Metrics*, IEEE Computer Society Press, 1997) editaram uma antologia de importantes artigos sobre métricas de software.

Métodos para estabelecer um programa de métricas e seus princípios subjacentes para medidas de software são considerados por Ebert e seus colegas (*Best Practices in Software Measurement*, Springer, 2004). Shepperd (*Foundations of Software Measurement*, Prentice-Hall, 1996) também trata da teoria de medidas com algum detalhe. As pesquisas atuais são apresentadas no *Proceedings of the Symposium on Software Metrics* (IEEE, publicado anualmente).

Um resumo abrangente de dezenas de métricas úteis de software é apresentado na [IEE93]. Em geral, foi resumida uma discussão de cada uma das métricas com as “primitivas” (medidas) essenciais necessárias para computar a métrica e as relações apropriadas para efetuar a computação. Um apêndice fornece discussão e muitas referências.

Whitmire [Whi97] apresenta um tratamento abrangente e matematicamente sofisticado das métricas orientadas a objeto. Lorenz e Kidd [Lor94] e Hendersen-Sellers (*Object-Oriented Metrics: Measures of Complexity*, Prentice-Hall, 1996) realizam tratamentos dedicados às métricas orientadas a objeto.

Há disponível na Internet uma grande variedade de fontes de informação sobre métricas de software. Uma lista atualizada das referências na Web, relevantes às métricas de software, pode ser encontrada no site www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm.

GERENCIAMENTO DE PROJETOS DE SOFTWARE

Nesta parte de *Engenharia de Software: Uma Abordagem Profissional* você aprenderá as técnicas de gerenciamento necessárias para planejar, organizar, monitorar e controlar projetos de software. Estas questões são tratadas nos capítulos que seguem:

- Como as pessoas, os processos e os problemas devem ser gerenciados durante um projeto de software?
- Como as métricas de software podem ser usadas para gerenciar um projeto de software e o processo de software?
- Como uma equipe de software pode gerar estimativas confiáveis de trabalho, custo e duração do projeto?
- Que técnicas podem ser usadas para avaliar os riscos que podem ter um impacto sobre o sucesso do projeto?
- Como um gerente de projeto de software seleciona um conjunto de tarefas de engenharia de software?
- Como é criado um cronograma de projeto?
- Por que a manutenção e a reengenharia são tão importantes para os gerentes de engenharia de software e os profissionais?

Uma vez respondidas essas questões, você estará mais bem preparado para gerenciar projetos de software de forma a obter um produto de alta qualidade entregue no prazo.